

Algorithmique et programmation impérative

Chapitre 8 : Prouver les propriétés des algorithmes itératifs

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 8. Prouver les propriétés des algorithmes itératifs

- 1 Une élection étrange
- 2 Preuve d'exactitude d'un algorithme itératif
- 3 Exemple 1 (boucle while) : la division euclidienne
- 4 Exemple 2 (boucle while) : calcul de l'exposant
- 5 Exemple 3 (liste et invariant graphique) : maximum d'une liste
- 6 Exemple 4 (boucle for) : somme d'une séquence
- 7 Exemple 5 (boucle for) : ajout des carrés
- 8 Exemple 6 (plusieurs boucles imbriquées) : tri par sélection

Preuves des propriétés des algorithmes

Etant donné un (morceau) d'algorithme, on désire prouver que si les **préconditions** sont respectées, alors :

- **l'algorithme s'arrête** (prouver qu'il n'y a pas de boucle ou de récursion infinie, que l'algorithme s'arrête toujours) ;
- **l'algorithme est exact** (prouver que l'algorithme fait bien ce qu'il est supposé faire, que les **postconditions** sont respectées).

Dans ce chapitre (et ce cours) :

- nous allons présenter quelques algorithmes et prouver leurs propriétés (arrêt et exactitude) ;
- on se limite aux algorithmes **itératifs**. Les preuves concernant les algorithmes **récurifs** seront couvertes dans le cours de *Structures de Données I*.

Une élection étrange

Election à Weirdland

Deux candidats se sont présentés à l'élection présidentielle de Weirdland : le candidat A et le candidat B . Les bulletins de vote ont été rassemblés dans une urne¹ et les assesseurs s'apprêtent à dépouiller les résultats selon la méthode en vigueur dans leur pays, définie comme suit.

1. On considère qu'un bulletin contient soit A , soit B et qu'il n'y a pas de bulletin blanc ou nul.

Election à Weirdland

Deux candidats se sont présentés à l'élection présidentielle de Weirdland : le candidat A et le candidat B . Les bulletins de vote ont été rassemblés dans une urne¹ et les assesseurs s'apprêtent à dépouiller les résultats selon la méthode en vigueur dans leur pays, définie comme suit.

« Tant qu'il reste plus d'un seul bulletin de vote dans l'urne : prendre deux bulletins de vote au hasard dans l'urne ; si les deux bulletins tirés sont en faveur du même candidat, les jeter et ajouter un nouveau bulletin pour A (disponibles en quantité infinie) ; sinon (un bulletin A et un bulletin B), jeter le bulletin en faveur de A et remplacer le bulletin B dans l'urne. »

1. On considère qu'un bulletin contient soit A , soit B et qu'il n'y a pas de bulletin blanc ou nul.

Election à Weirdland

Durant cette procédure étrange, s'il ne reste qu'un seul bulletin, alors ce bulletin correspond au gagnant de l'élection.

La fonction `depouillement` (voir slide suivant) simule la procédure.

Elle prend en entrée une liste (Python) ne contenant que des 'A' ou des 'B' (précondition).

Election à Weirdland

```
import random

def depouillement(urne):
    while len(urne) > 1:
        print('Urne au début de l\'itération:', urne, end=' ')
        random.shuffle(urne)
        print(' -> ', urne)
        vot1 = urne.pop()
        vote2 = urne.pop()
        print("Votes retirés :", vot1, vote2, end=' ')
        if vot1 == vote2:
            print('(identiques: A ajouté)')
            urne.append('A')
        else:
            print('(différents: B ajouté)')
            urne.append('B')
    print('Urne à la fin de l\'itération:', urne, '\n', ('-' * 30))
```

(La fonction `random.shuffle` permet de réordonner aléatoirement les éléments d'une liste passée en paramètre.)

Election à Weirland

Par exemple, voici une exécution possible de dépouillement (urne) si urne référence la liste ['A', 'A', 'B', 'B', 'A'] :

```
Urne au début de l'itération: ['A', 'A', 'B', 'B', 'A'] -> ['B', 'A', 'B', 'A', 'A']
Votes retirés : A A (identiques: A ajouté)
Urne à la fin de l'itération: ['B', 'A', 'B', 'A']
-----
Urne au début de l'itération: ['B', 'A', 'B', 'A'] -> ['A', 'B', 'B', 'A']
Votes retirés : A B (différents: B ajouté)
Urne à la fin de l'itération: ['A', 'B', 'B']
-----
Urne au début de l'itération: ['A', 'B', 'B'] -> ['A', 'B', 'B']
Votes retirés : B B (identiques: A ajouté)
Urne à la fin de l'itération: ['A', 'A']
-----
Urne au début de l'itération: ['A', 'A'] -> ['A', 'A']
Votes retirés : A A (identiques: A ajouté)
Urne à la fin de l'itération: ['A']
```

Selon cette procédure, A serait donc déclaré vainqueur.

Election à Weirdland

Les habitants de Weirdland prétendent que ce type de dépouillement se termine toujours.

Election à Weirdland

Les habitants de Weirdland prétendent que ce type de dépouillement se termine toujours.

De plus, et bien que la procédure est en partie aléatoire, certains affirment qu'il existe une règle déterministe permettant de connaître le candidat qui est déclaré vainqueur. La règle serait la suivante :

Election à Weirdland

Les habitants de Weirdland prétendent que ce type de dépouillement se termine toujours.

De plus, et bien que la procédure est en partie aléatoire, certains affirment qu'il existe une règle déterministe permettant de connaître le candidat qui est déclaré vainqueur. La règle serait la suivante :

« Si le nombre de bulletins pour B est pair, alors A est élu. Sinon, B est élu. » (★)

Election à Weirdland

Si on suppose que le paramètre `urne` référence une liste non vide ne contenant que des 'A' et des 'B', est-il possible de prouver que

- a) la fonction `depouillement` se termine toujours ?
- b) si la liste `urne` contient initialement x 'B' et que si l'on note par y le nombre de 'B' courant dans la liste (entre 2 itérations de la boucle), alors l'assertion

$$S \equiv x \text{ et } y \text{ ont la même parité}$$

est vraie

- ▶ avant d'entrer dans la boucle ; et
 - ▶ après chaque itération **complète** de la boucle ?
- c) si les assertions ci-dessus sont prouvées, alors la règle (*), énoncée par les habitants de Weirdland, est toujours vraie ?

Election à Weirdland

Si on suppose que le paramètre `urne` référence une liste non vide ne contenant que des 'A' et des 'B', est-il possible de prouver que

- a) la fonction `depouillement` se termine toujours ?
- b) si la liste `urne` contient initialement x 'B' et que si l'on note par y le nombre de 'B' courant dans la liste (entre 2 itérations de la boucle), alors l'assertion

$$S \equiv x \text{ et } y \text{ ont la même parité}$$

est vraie

- ▶ avant d'entrer dans la boucle ; et
- ▶ après chaque itération complète de la boucle ?
- c) si les assertions ci-dessus sont prouvées, alors la règle ($\star$), énoncée par les habitants de Weirdland, est toujours vraie ?

Election à Weirdland

Si on suppose que le paramètre `urne` référence une liste non vide ne contenant que des 'A' et des 'B', est-il possible de prouver que

- a) la fonction `depouillement` se termine toujours ?
- b) si la liste `urne` contient initialement x 'B' et que si l'on note par y le nombre de 'B' courant dans la liste (entre 2 itérations de la boucle), alors l'assertion

$$S \equiv x \text{ et } y \text{ ont la même parité}$$

est vraie

- ▶ avant d'entrer dans la boucle ; et
 - ▶ après chaque itération complète de la boucle ?
- c) si les assertions ci-dessus sont prouvées, alors la règle ($\star$), énoncée par les habitants de Weirdland, est toujours vraie ?

Election à Weirdland

Si on suppose que le paramètre `urne` référence une liste non vide ne contenant que des 'A' et des 'B', est-il possible de prouver que

- a) la fonction `depouillement` se termine toujours ?
- b) si la liste `urne` contient initialement x 'B' et que si l'on note par y le nombre de 'B' courant dans la liste (entre 2 itérations de la boucle), alors l'assertion

$$S \equiv x \text{ et } y \text{ ont la même parité}$$

est vraie

- ▶ avant d'entrer dans la boucle ; et
 - ▶ après chaque itération complète de la boucle ?
- c) si les assertions ci-dessus sont prouvées, alors la règle ($\star$), énoncée par les habitants de Weirdland, est toujours vraie ?

Election à Weirdland

a) La fonction `depouillement` se termine-t-elle toujours ?

```
def depouillement(urne):  
    while len(urne) > 1:  
        ...  
        vote1 = urne.pop()  
        vote2 = urne.pop()  
        print("Votes retirés :", vote1, vote2, end=' ')  
        if vote1 == vote2:  
            print('(identiques: A ajouté)')  
            urne.append('A')  
        else:  
            print('(différents: B ajouté)')  
            urne.append('B')
```

- A chaque itération, deux votes sont retirés et, quelque soit le résultat du `if`, un vote est rajouté ;
- Il y a donc exactement un bulletin en moins à chaque itération ;
- La boucle s'arrêtera quand il n'y aura plus qu'un bulletin.

Election à Weirland

b) L'assertion " $S \equiv x$ et y ont la même parité" est-elle vraie avant et après chaque itération ?

```
def depouillement(urne):
    while len(urne) > 1:
        ...
        vote1 = urne.pop()
        vote2 = urne.pop()
        print("Votes retirés :", vote1, vote2, end=' ')
        if vote1 == vote2:
            print(' (identiques: A ajouté)')
            urne.append('A')
        else:
            print(' (différents: B ajouté)')
            urne.append('B')
```

- Avant d'entrer dans la boucle, $x = y$ puisque rien n'a été modifié (x est le nombre de 'B' au départ et y le nombre de 'B' courant)
- Il y a trois cas possibles :
 - ▶ Si on tire deux 'B', alors on ajoute un 'A' et y est diminué de 2
 - ▶ Si on tire deux 'A', alors on ajoute un 'A' et y est inchangé
 - ▶ Si on tire un 'A' et un 'B', alors on ajoute un 'B' et y est inchangé
- Dans tous les cas l'assertion reste vraie si elle était vraie à l'itération précédente. Par induction, S est vraie.

Election à Weirdland

c) Grâce à a) et b), peut-on affirmer que la règle $(\star)$, énoncée par les habitants de Weirdland, est toujours vraie ?

« Si le nombre de bulletins pour B est pair, alors A est élu. Sinon, B est élu. » $(\star)$

- On sait grâce à a) que la procédure s'arrête quand il n'y a plus qu'un seul bulletin ;
- On sait grâce à b) que s'il reste un seul bulletin pour 'B' à la fin, et donc un nombre impair, cela signifie qu'il y avait un nombre impair de bulletins pour 'B' au départ ;
- Ces deux éléments suffisent pour conclure que $(\star)$ est vraie.

Preuve d'exactitude d'un algorithme itératif

Preuve d'exactitude d'un algorithme itératif

La preuve complète qu'un algorithme itératif est exact revient à :

- montrer qu'il s'arrête ;
- à exhiber, pour chaque boucle, une propriété (**invariant de boucle**) qui, si elle est valide avant l'exécution d'un tour de boucle, est aussi valide après l'exécution du tour de boucle ; et la prouver
- vérifier que les conditions initiales rendent la propriété vraie en entrée du premier tour de boucle
- conclure que cette propriété est vraie en sortie du dernier tour de boucle ; un bon choix de la propriété prouvera qu'on a bien produit le résultat souhaité.

Preuve d'exactitude d'un algorithme itératif

La preuve complète qu'un algorithme itératif est exact revient à :

- montrer qu'il s'arrête ;
- à exhiber, pour chaque boucle, une propriété (**invariant de boucle**) qui, si elle est valide avant l'exécution d'un tour de boucle, est aussi valide après l'exécution du tour de boucle ; et la prouver
- vérifier que les conditions initiales rendent la propriété vraie en entrée du premier tour de boucle
- conclure que cette propriété est vraie en sortie du dernier tour de boucle ; un bon choix de la propriété prouvera qu'on a bien produit le résultat souhaité.

Preuve d'exactitude d'un algorithme itératif

La preuve complète qu'un algorithme itératif est exact revient à :

- montrer qu'il s'arrête ;
- à exhiber, pour chaque boucle, une propriété (**invariant de boucle**) qui, si elle est valide avant l'exécution d'un tour de boucle, est aussi valide après l'exécution du tour de boucle ; et la prouver
- vérifier que les conditions initiales rendent la propriété vraie en entrée du premier tour de boucle
- conclure que cette propriété est vraie en sortie du dernier tour de boucle ; un bon choix de la propriété prouvera qu'on a bien produit le résultat souhaité.

Preuve d'exactitude d'un algorithme itératif

La preuve complète qu'un algorithme itératif est exact revient à :

- montrer qu'il s'arrête ;
- à exhiber, pour chaque boucle, une propriété (**invariant de boucle**) qui, si elle est valide avant l'exécution d'un tour de boucle, est aussi valide après l'exécution du tour de boucle ; et la prouver
- vérifier que les conditions initiales rendent la propriété vraie en entrée du premier tour de boucle
- conclure que cette propriété est vraie en sortie du dernier tour de boucle ; un bon choix de la propriété prouvera qu'on a bien produit le résultat souhaité.

Preuve d'exactitude d'un algorithme itératif

La preuve complète qu'un algorithme itératif est exact revient à :

- montrer qu'il s'arrête ;
- à exhiber, pour chaque boucle, une propriété (**invariant de boucle**) qui, si elle est valide avant l'exécution d'un tour de boucle, est aussi valide après l'exécution du tour de boucle ; et la prouver
- vérifier que les conditions initiales rendent la propriété vraie en entrée du premier tour de boucle
- conclure que cette propriété est vraie en sortie du dernier tour de boucle ; un bon choix de la propriété prouvera qu'on a bien produit le résultat souhaité.

La principale difficulté de ce type de preuve réside dans la détermination de l'invariant de boucle.

Preuve d'arrêt d'un algorithme itératif

Pour prouver qu'un algorithme itératif s'arrête (si les préconditions sont respectées), il faut prouver que **chaque boucle se termine en un nombre fini d'itérations**.

Remarque : c'est en général facile à montrer (voir par ex. preuve pour `countdown` au ch. 6) mais il y a certains rares cas où cela peut être très difficile (voir chap. 6 et la conjecture de Syracuse).

Invariant de boucle

Intuitivement, un **invariant de boucle** est une assertion (une propriété) qui

- est vraie avant l'entrée dans la boucle ;
- est toujours vraie **après** chaque itération de la boucle.

Remarques :

- cela implique que si une boucle s'arrête, l'invariant est toujours vrai après l'exécution complète de celle-ci ;
- l'invariant ne doit pas être vérifié à tout moment dans le corps de la boucle, mais *à la fin* de chaque itération de celle-ci.

Invariant de boucle

Plus formellement, on désire qu'une propriété $\mathcal{P}$ soit vérifiée à la fin de l'exécution d'une boucle.

Définition

Un **invariant de boucle** est une **propriété** Q qui est

- vraie **avant** d'entrer dans la boucle (avant le 1^{er} passage) ;
- reste vraie **après chaque** passage dans la boucle ;
- vaut la propriété désirée $\mathcal{P}$ à la **fin** de l'exécution complète de la boucle.

Exemple 1 (boucle while) : la division euclidienne

Algorithme : division euclidienne

Problème

Soit a et b deux entiers tels que $a \geq 0$ et $b > 0$. Comment déterminer le quotient (entier) et le reste de la division de a par b ?

Idée de l'algorithme de division euclidienne :

Initialiser le reste à a . Ensuite, soustraire b au reste tant que celui-ci est plus grand que b . Le nombre de soustractions est égal au quotient.

Exemple : Soit $a = 14$ et $b = 5$.

# soustractions	reste
0	14
1	9
2	4

Le quotient de a par b est 2 et le reste est 4.

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

>>> division(14, 5)
(2, 4)

```

A prouver : si a et b sont deux entiers tels que $a \geq 0$ et $b > 0$,

- la fonction `division` s'arrête-t-elle toujours ? (preuve de l'arrêt)
- après l'exécution de la fonction `division` les valeurs retournées correspondent-elles bien au quotient de a par b et au reste ? (preuve de l'exactitude)

Division euclidienne : preuve d'arrêt

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Preuve d'arrêt.

Soient $a \geq 0$ et $b > 0$.

- *Base.* Soit $a < b$. Dans ce cas, comme r est initialisé à a , le test (4) de la boucle est faux et la boucle n'est jamais exécutée.
- *Gén.* Soit $a \geq b$.
 - ▶ Par récurrence, supposons que la boucle s'arrête pour tout $r \leq k$ ($k \geq 0$).
 - ▶ La boucle s'arrête également pour $r = k + 1$ car l'instruction (5) diminue strictement la valeur de r (car $b > 0$ et b n'est jamais modifié dans la boucle). □

Remarque : notez l'importance de l'hypothèse $b > 0$ dans cette preuve !

Division euclidienne : preuve d'arrêt

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Preuve d'arrêt.

Soient $a \geq 0$ et $b > 0$.

- *Base.* Soit $a < b$. Dans ce cas, comme r est initialisé à a , le test (4) de la boucle est faux et la boucle n'est jamais exécutée.
- *Gén.* Soit $a \geq b$.
 - ▶ Par récurrence, supposons que la boucle s'arrête pour tout $r \leq k$ ($k \geq 0$).
 - ▶ La boucle s'arrête également pour $r = k + 1$ car l'instruction (5) diminue strictement la valeur de r (car $b > 0$ et b n'est jamais modifié dans la boucle). □

Remarque : notez l'importance de l'hypothèse $b > 0$ dans cette preuve !

Division euclidienne : invariant de boucle

Nous allons montrer que l'assertion suivante est un **invariant** de la boucle `while` :

$$a = b \times q + r.$$

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

a	b	q	r	$b \times q + r$
14	5	0	14	$5 \times 0 + 14$
14	5	1	9	$5 \times 1 + 9$
14	5	2	4	$5 \times 2 + 4$

$a = b \times q + r$ est-il un invariant de boucle ?

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

- Avant d'entrer dans la boucle : q est initialisé à 0 et r à a , donc $b \times q + r = b \times 0 + a = a$ et l'assertion est vraie.
- il reste à prouver que cela reste toujours vrai **après** une itération de la boucle.

Considérons une itération donnée de la boucle et vérifions que l'assertion reste vraie **à la fin de cette itération**.

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Les variables q et r étant modifiées dans la boucle, on peut utiliser les notations suivantes :

- q_0 et r_0 : valeurs de q et r **au début** de l'itération ;
- q_1 et r_1 : valeurs de q et r **à la fin** de l'itération.

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a = b \times q_0 + r_0$
- On a $r_0 = r_1 + b$ et $q_0 = q_1 - 1$ (instructions 5 et 6)
- Donc, $a = b \times q_0 + r_0 = b \times (q_1 - 1) + r_1 + b$
- Après simplification, on obtient $a = b \times q_1 + r_1$
- L'assertion reste donc vraie après l'itération considérée et $a = b \times q + r$ est bien un invariant de la boucle while $\square$

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a = b \times q_0 + r_0$
- On a $r_0 = r_1 + b$ et $q_0 = q_1 - 1$ (instructions 5 et 6)
- Donc, $a = b \times q_0 + r_0 = b \times (q_1 - 1) + r_1 + b$
- Après simplification, on obtient $a = b \times q_1 + r_1$
- L'assertion reste donc vraie après l'itération considérée et $a = b \times q + r$ est bien un invariant de la boucle while $\square$

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a = b \times q_0 + r_0$
- On a $r_0 = r_1 + b$ et $q_0 = q_1 - 1$ (instructions 5 et 6)
- Donc, $a = b \times q_0 + r_0 = b \times (q_1 - 1) + r_1 + b$
- Après simplification, on obtient $a = b \times q_1 + r_1$
- L'assertion reste donc vraie après l'itération considérée et $a = b \times q + r$ est bien un invariant de la boucle while $\square$

```

1  def division(a, b):
2      r = a
3      q = 0
4      while r >= b:
5          r = r - b
6          q = q + 1
7      return (q, r)

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a = b \times q_0 + r_0$
- On a $r_0 = r_1 + b$ et $q_0 = q_1 - 1$ (instructions 5 et 6)
- Donc, $a = b \times q_0 + r_0 = b \times (q_1 - 1) + r_1 + b$
- Après simplification, on obtient $a = b \times q_1 + r_1$
- L'assertion reste donc vraie après l'itération considérée et $a = b \times q + r$ est bien un invariant de la boucle while $\square$

Division euclidienne : preuve d'exactitude

La preuve **complète** de l'**exactitude** de la fonction `division` est donc :

- preuve de l'arrêt (voir ci-avant)
- preuve que $a = b \times q + r$ est un invariant de la boucle (voir ci-avant)
- le résultat retourné est correct car :
 - ▶ la boucle se termine quand la condition est fausse, c-à-d, quand $r < b$ (= négation de la condition) ;
 - ▶ à ce moment là, $a = b \times q + r$ est vrai, donc, $q = \frac{a-r}{b}$ et $r < b$, ce qui correspond à la définition du quotient et du reste d'une division².

2. Ici nous utilisons le fait (sans le prouver, cf. cours de *Mathématiques élémentaires*) qu'il existe un couple unique d'entiers q et r tels que $q = \frac{a-r}{b}$ et $r < b$.

Exemple 2 (boucle while) : calcul de l'exposant

Algorithme : calcul de l'exposant

Problème

Soit a un nombre réel et n un entier ≥ 0 . Comment calculer a^n ?

- **Approche classique** : utiliser une boucle pour calculer $a \times a \times \dots \times a$, ce qui revient à faire $n - 1$ multiplications
- **Exponentielle rapide** : exploiter le fait que
 - ▶ si n est pair, alors $a^n = (a^2)^{\frac{n}{2}}$
 - ▶ si n est impair, alors $a^n = a \cdot a^{n-1}$

Exemple : $a^9 = a \cdot a^8 = a \cdot (a^2)^4 = a \cdot ((a^2)^2)^2$, ce qui revient à 4 multiplications au lieu de 8

On verra au chapitre 9 que la deuxième version est effectivement (beaucoup) plus rapide. Mais ici, dans ce chapitre, montrons déjà que l'algorithme va effectivement nous retourner a^n .

Algorithme : calcul de l'exposant

```
1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r
```

```
>>> expo(3,3)
```

```
27
```

```
>>> expo(2,0)
```

```
1
```

Remarque : b (resp. i) représente le a (resp. n) courant, puisque l'algorithme peut modifier ces valeurs. On pourrait se passer de ces variables et modifier a et n (voir version du ch. 9), mais ce sera plus facile pour l'analyse.

Division euclidienne : preuve d'arrêt

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Preuve d'arrêt.

Soit $n \geq 0$. Par récurrence sur n .

- *Base.* Si $n = 0$, la boucle n'est pas exécutée.
- *Gén.* Soit $n > 0$
 - ▶ Par récurrence, supposons que la boucle s'arrête pour tout $i \leq n - 1$;
 - ▶ Si i est pair, l'instruction (8) diminue strictement sa valeur ;
 - ▶ Si i est impair, l'instruction (11) diminue strictement sa valeur ;
 - ▶ Par induction, la boucle s'arrête donc également quand $i = n$. □

Division euclidienne : preuve d'arrêt

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Preuve d'arrêt.

Soit $n \geq 0$. Par récurrence sur n .

- *Base.* Si $n = 0$, la boucle n'est pas exécutée.
- *Gén.* Soit $n > 0$
 - ▶ Par récurrence, supposons que la boucle s'arrête pour tout $i \leq n - 1$;
 - ▶ Si i est pair, l'instruction (8) diminue strictement sa valeur ;
 - ▶ Si i est impair, l'instruction (11) diminue strictement sa valeur ;
 - ▶ Par induction, la boucle s'arrête donc également quand $i = n$. □

Calcul de l'exposant : invariant de boucle

Nous allons montrer que l'assertion suivante est un **invariant** de la boucle `while` :

$$a^n = r \cdot b^i.$$

Soit $a = 2$ et $n = 9$, alors $a^n = 512$.

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

r	b	i	$r \times b^i$
1	2	9	$1 \times 2^9 = 512$
2	2	8	$2 \times 2^8 = 512$
2	4	4	$2 \times 4^4 = 512$
2	16	2	$2 \times 16^2 = 512$
2	256	1	$2 \times 256^1 = 512$
512	256	0	$512 \times 256^0 = 512$

$a^n = r \cdot b^i$ est-il un invariant de boucle ?

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

- Avant d'entrer dans la boucle : l'initialisation des variables (instr. 2 à 4) donne $r \cdot b^i = 1 \cdot a^n = a^n$ et l'assertion est vraie.
- il reste à prouver que cela reste toujours vrai après une itération de la boucle.

Considérons une itération donnée de la boucle et vérifions que l'assertion reste vraie **à la fin de cette itération**.

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Les variables a et n ne sont pas modifiées par la boucle. On va utiliser les notations i_0 , b_0 et r_0 pour les valeurs des variables i , b et r **au début** de l'itération et i_1 , b_1 et r_1 pour leurs valeurs **à la fin** de l'itération.

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a^n = r_0 \cdot b_0^{i_0}$
- Si i_0 est pair, alors
 - ▶ les instr. 7 et 8 donnent $b_0 = \sqrt{b_1}$, $i_0 = 2i_1$ et $r_0 = r_1$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = r_1 \cdot \sqrt{b_1}^{2i_1} = r_1 \cdot b_1^{\frac{1}{2} \cdot 2i_1} = r_1 \cdot b_1^{i_1}.$$
- Si i_0 est impair, alors
 - ▶ les instr. 10 et 11 donnent $b_0 = b_1$, $i_0 = i_1 + 1$ et $r_0 = \frac{r_1}{b_1}$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = \frac{r_1}{b_1} \cdot b_1^{(i_1+1)} = \frac{r_1}{b_1} \cdot b_1 \cdot b_1^{i_1} = r_1 \cdot b_1^{i_1}.$$
- Quelle que soit la parité de i_0 , l'assertion reste vraie après l'itération considérée et $a^n = r \cdot b^i$ est donc bien un invariant de la boucle $\square$

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a^n = r_0 \cdot b_0^{i_0}$
- Si i_0 est pair, alors
 - ▶ les instr. 7 et 8 donnent $b_0 = \sqrt{b_1}$, $i_0 = 2i_1$ et $r_0 = r_1$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = r_1 \cdot \sqrt{b_1}^{2i_1} = r_1 \cdot b_1^{\frac{1}{2} \cdot 2i_1} = r_1 \cdot b_1^{i_1}.$$
- Si i_0 est impair, alors
 - ▶ les instr. 10 et 11 donnent $b_0 = b_1$, $i_0 = i_1 + 1$ et $r_0 = \frac{r_1}{b_1}$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = \frac{r_1}{b_1} \cdot b_1^{(i_1+1)} = \frac{r_1}{b_1} \cdot b_1 \cdot b_1^{i_1} = r_1 \cdot b_1^{i_1}.$$
- Quelle que soit la parité de i_0 , l'assertion reste vraie après l'itération considérée et $a^n = r \cdot b^i$ est donc bien un invariant de la boucle $\square$

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a^n = r_0 \cdot b_0^{i_0}$
- Si i_0 est pair, alors
 - ▶ les instr. 7 et 8 donnent $b_0 = \sqrt{b_1}$, $i_0 = 2i_1$ et $r_0 = r_1$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = r_1 \cdot \sqrt{b_1}^{2i_1} = r_1 \cdot b_1^{\frac{1}{2} \cdot 2i_1} = r_1 \cdot b_1^{i_1}.$$
- Si i_0 est impair, alors
 - ▶ les instr. 10 et 11 donnent $b_0 = b_1$, $i_0 = i_1 + 1$ et $r_0 = \frac{r_1}{b_1}$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = \frac{r_1}{b_1} \cdot b_1^{(i_1+1)} = \frac{r_1}{b_1} \cdot b_1 \cdot b_1^{i_1} = r_1 \cdot b_1^{i_1}.$$
- Quelle que soit la parité de i_0 , l'assertion reste vraie après l'itération considérée et $a^n = r \cdot b^i$ est donc bien un invariant de la boucle $\square$

```

1  def expo(a, n):
2      r = 1
3      b = a
4      i = n
5      while i > 0:
6          if i % 2 == 0:
7              b = b * b
8              i = i // 2
9          else:
10             r = r * b
11             i = i - 1
12     return r

```

Prouvons que l'assertion est toujours vraie après une itération donnée :

- Par hypothèse d'induction : $a^n = r_0 \cdot b_0^{i_0}$
- Si i_0 est pair, alors
 - ▶ les instr. 7 et 8 donnent $b_0 = \sqrt{b_1}$, $i_0 = 2i_1$ et $r_0 = r_1$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = r_1 \cdot \sqrt{b_1}^{2i_1} = r_1 \cdot b_1^{\frac{1}{2} \cdot 2i_1} = r_1 \cdot b_1^{i_1}.$$
- Si i_0 est impair, alors
 - ▶ les instr. 10 et 11 donnent $b_0 = b_1$, $i_0 = i_1 + 1$ et $r_0 = \frac{r_1}{b_1}$
 - ▶ Donc,

$$a^n = r_0 \cdot b_0^{i_0} = \frac{r_1}{b_1} \cdot b_1^{(i_1+1)} = \frac{r_1}{b_1} \cdot b_1 \cdot b_1^{i_1} = r_1 \cdot b_1^{i_1}.$$
- Quelle que soit la parité de i_0 , l'assertion reste vraie après l'itération considérée et $a^n = r \cdot b^i$ est donc bien un invariant de la boucle $\square$

Calcul de l'exposant : preuve d'exactitude

La preuve **complète** de l'**exactitude** de la fonction `expo` est donc :

- preuve de l'arrêt (voir ci-avant) ;
- preuve que $a^n = r \cdot b^i$ est un invariant de la boucle (voir ci-avant) ;
- le résultat retourné (valeur de r) est correct car :
 - ▶ quand la boucle se termine, on a $i = 0$;
 - ▶ après la boucle, l'invariant devient donc :

$$r = \frac{a^n}{b^i} = \frac{a^n}{b^0} = \frac{a^n}{1} = a^n;$$

- ▶ la valeur retournée est donc bien a^n .

Exemple 3

(liste et invariant graphique) :
maximum d'une liste

Algorithme de parcours d'une liste : maximum d'une liste

La notation indicée pour les variables modifiées dans une boucle n'est pas toujours bien adaptée à certains invariants.

Exemple : connaître la valeur maximum contenue dans une liste d'entiers.

Algorithme de parcours d'une liste : maximum d'une liste

```
def maximum(A):  
    if len(A) == 0:  
        return None  
    maxCourant = A[0]  
    i = 1  
    n = len(A)  
    while i < n:  
        if A[i] > maxCourant:  
            maxCourant = A[i]  
        i = i + 1  
    return maxCourant  
  
max = maximum([7, 8, 4, 2, 65, 7, 3, 17])  
print('La plus grande valeur est', max)
```

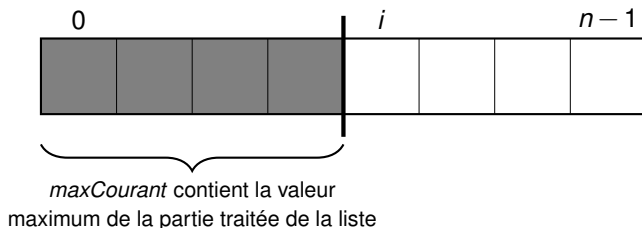
Preuve graphique

Dans cet exemple (et pour beaucoup d'algorithmes parcourant une liste), il est plus pratique de **représenter l'invariant graphiquement** et d'en expliquer la preuve directement via ce **dessin**.

Preuve graphique

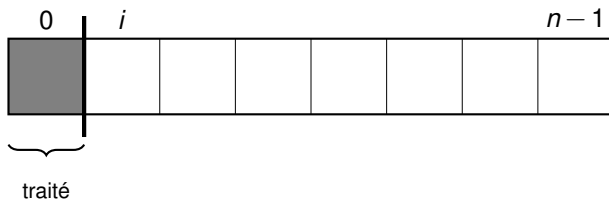
Dans cet exemple (et pour beaucoup d'algorithmes parcourant une liste), il est plus pratique de **représenter l'invariant graphiquement** et d'en expliquer la preuve directement via ce **dessin**.

Invariant (propriété Q) :



Avant d'entrer dans la boucle

```
maxCourant = A[0]  
i = 1  
n = len(A)  
...
```



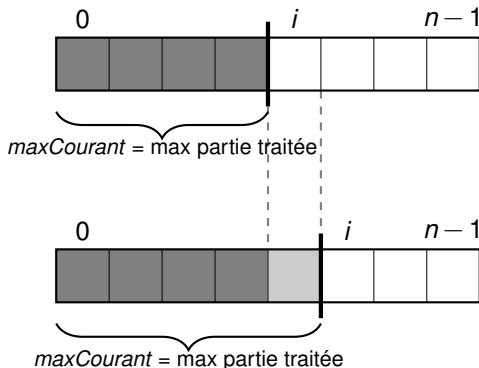
maxCourant contient bien la valeur maximum de la partie traitée de la liste.

Après chaque nouvelle itération

On suppose l'invariant vrai
après k itérations (ici $k = i - 1$).

Cette hypothèse, ainsi que le
corps de la boucle, assurent
qu'après l'itération suivante,
 Q est toujours vérifiée.

```
...
while i < n:
    if A[i] > maxCourant:
        maxCourant = A[i]
    i = i + 1
...
```



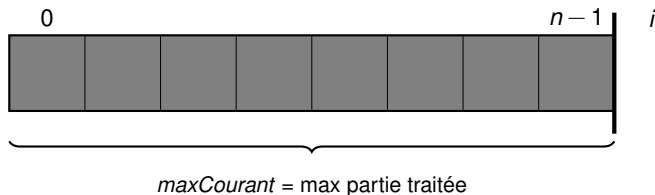
Après l'exécution complète de la boucle

```

...
while i < n:
    ...
return maxCourant

```

A la fin, $i = n$ et la propriété Q vérifiée précédemment devient :



Ceci conduit à la propriété $\mathcal{P}$ désirée.

Exemple 4 (boucle for) : somme d'une séquence

Algorithme avec une boucle for : preuve d'arrêt

Prouver qu'une boucle `for` s'arrête est très simple si celle-ci est appliquée sur une séquence respectant les 3 conditions suivantes :

- la boucle itère sur chaque élément de la séquence (une et une seule fois) ;
- la séquence est finie ;
- le corps de la boucle ne modifie pas le nombre d'éléments de la séquence.

En effet, sous ces hypothèses il est évident que la boucle se termine en un nombre fini d'étapes.

Somme d'une séquence : preuve d'arrêt

En Python, les deux premières conditions sont automatiquement gérées par la syntaxe d'une boucle `for` et par le fait qu'une séquence est toujours finie.

Donc, si le corps de la boucle ne modifie pas (n'augmente pas) le nombre d'éléments de la séquence, la preuve de l'arrêt est immédiate.

```
1  def sum_all(liste):
2      res = 0
3      for item in liste:
4          res += item
5      return res
```

Remarque : pour le moment nous travaillons avec des séquences finies, donc ce qui est dit ici est correct, **mais** il existe une notion de **générateurs** qui permettent de générer des séquences infinies. Dans ce cas, la preuve d'arrêt doit être faite.

Somme d'une séquence : invariant de boucle

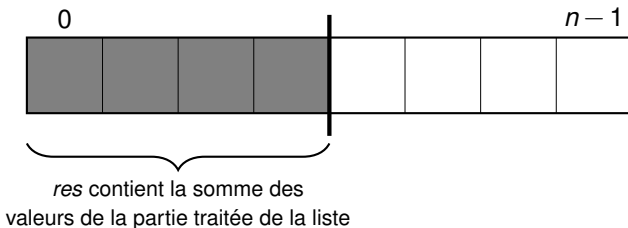
Le fait qu'une boucle `for` s'applique linéairement sur une séquence rend l'utilisation d'une preuve graphique aisée.

```

1  def sum_all(liste):
2      res = 0
3      for item in liste:
4          res += item
5      return res

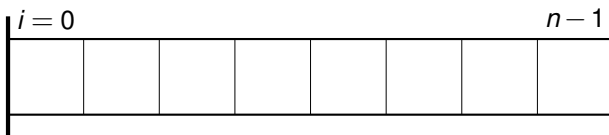
```

Invariant (propriété Q) :



Avant d'entrer dans la boucle

```
1  def sum_all(liste):  
2      res = 0  
3      for item in liste:  
4          res += item  
5      return res
```



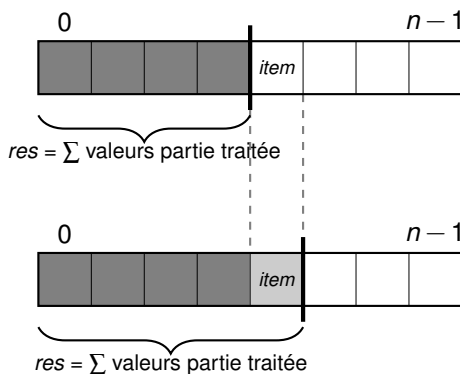
La partie traitée est vide, et l'invariant est donc trivialement vrai.

Après chaque nouvelle itération

```

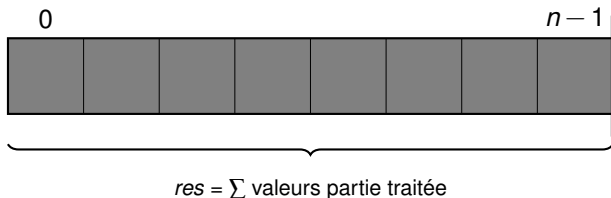
1  def sum_all(liste):
2      res = 0
3      for item in liste:
4          res += item
5      return res

```



Après l'exécution complète de la boucle

A la fin, la boucle `for` a parcouru toute la liste et la propriété Q vérifiée précédemment devient :



Ceci conduit à la propriété $\mathcal{P}$ désirée.

Exemple 5 (boucle for) : ajout des carrés

Preuve d'arrêt d'un algorithme itératif : boucle for

La fonction suivante prend une liste d'entiers en argument **mais augmente la taille de la liste** en ajoutant après chaque élément son carré.

```
1 def add_square(liste):
2     index = 0
3     for item in liste:
4         if index % 2 == 0:
5             liste.insert(index+1, item*item)
6             index = index + 1
```

```
>>> t = [1, 2, 3]
>>> add_square(t)
>>> t
[1, 1, 2, 4, 3, 9]
```

Ici une preuve d'arrêt serait nécessaire, mais cette fonction peut être réécrite de telle sorte qu'une preuve est inutile (voir slide suivant). Ce genre de code est à éviter !

Preuve d'arrêt d'un algorithme itératif : boucle for

On construit une nouvelle liste et la preuve devient inutile. On retourne la liste modifiée plutôt que de la transformer dans la fonction.

```

1  def add_square(liste):
2      res = []
3      for item in liste:
4          res.extend([item, item * item])
5      return res

```

```

>>> t = [1, 2, 3]
>>> t = add_square(t)
>>> t
[1, 1, 2, 4, 3, 9]

```

La méthode `t1.extend(t2)` permet d'ajouter tous les éléments d'une liste `t2` à la fin de la liste `t1`.

Exemple 6 (plusieurs boucles imbriquées) : tri par sélection

Algorithme : tri par sélection

Problème

Soit une séquence de n entiers ($n \geq 0$), comment trier (par ordre croissant) les éléments de la séquence ?

Idée du tri par sélection.

Imaginons que nous devons trier une main d'un jeu de cartes. On pose les cartes faces visibles sur la table. On sélectionne la plus “petite” carte et on la prend en main. On sélectionne ensuite la plus petite carte parmi celles restées sur la table et on la place à droite de la carte en main. On répète le processus jusqu'à avoir sélectionné toutes les cartes posées sur la table.



Algorithme : tri par sélection

Idée du tri par sélection.

Pour appliquer cette idée à une séquence d'entiers, on représente la séquence comme suit : la partie grisée est la partie triée de la séquence (les cartes en main) et la partie blanche représente la partie non triée (les cartes sur la table).

3	7	2	6	5	1	4
---	---	---	---	---	---	---

1	7	2	6	5	3	4
---	---	---	---	---	---	---

1	2	7	6	5	3	4
---	---	---	---	---	---	---

1	2	3	6	5	7	4
---	---	---	---	---	---	---

1	2	3	4	5	7	6
---	---	---	---	---	---	---

1	2	3	4	5	7	6
---	---	---	---	---	---	---

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Algorithme : tri par sélection

Idée du tri par sélection.

Pour appliquer cette idée à une séquence d'entiers, on représente la séquence comme suit : la partie grisée est la partie triée de la séquence (les cartes en main) et la partie blanche représente la partie non triée (les cartes sur la table).

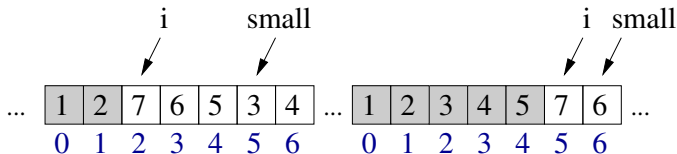
3	7	2	6	5	1	4
1	7	2	6	5	3	4
1	2	7	6	5	3	4
1	2	3	6	5	7	4
1	2	3	4	5	7	6
1	2	3	4	5	7	6
1	2	3	4	5	6	7

- sélectionner une valeur revient à échanger la première valeur non triée avec la plus petite valeur non triée (avec les cartes cela revient à les trier par échanges successifs en les laissant sur la table)
- quand il ne reste plus qu'un élément à trier, on peut s'arrêter

Algorithme : tri par sélection

On va utiliser deux boucles imbriquées :

- Une première boucle permet de sélectionner chaque élément un par un : quand l'indice de la première boucle vaut i , les i premiers éléments sont triés (indices 0 à $i - 1$). A chaque itération de cette boucle, on déplace le plus petit élément de la partie non triée, pour élargir d'une case la partie triée.
- Une deuxième boucle (interne à la première) est donc nécessaire pour déterminer l'indice de la plus petite valeur des éléments non triés (*small*).



Algorithme : tri par sélection

```
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])
```

```
>>> t = [3, 7, 2, 6, 5, 1, 4]
>>> selection_sort(t)
>>> t
[1, 2, 3, 4, 5, 6, 7]
```

Tri par sélection : preuve d'arrêt

S'il y a plusieurs boucles, il faut prouver que chacune d'elles s'arrête.

```
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])
```

Preuve.

On observe d'abord que, sous l'hypothèse qu'un indice $i < n - 1$ est fixé, la boucle interne (instr. 5 – 7) s'arrête, car elle s'applique sur une séquence finie. Ensuite, on observe que la boucle externe (instr. 3 – 8) s'applique sur une séquence finie et s'arrête également. □

Tri par sélection : invariants de boucles

```

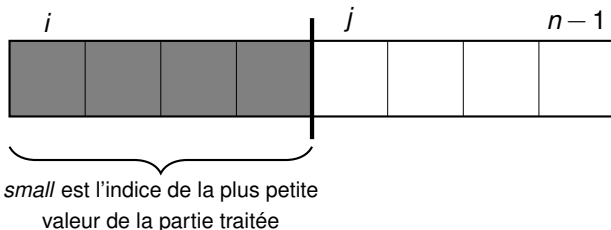
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])

```

Invariants des boucles ?

- **Boucle interne** : La valeur de *small* est l'indice du plus petit élément de la sous-séquence $t[i], \dots, t[j-1]$
- **Boucle externe** : La partie traitée (indices de 0 à $i-1$) est **triée** et toutes ses valeurs sont **inférieures ou égales** aux valeurs de la partie non-traitée.

Tri par sélection : invariant de la boucle interne

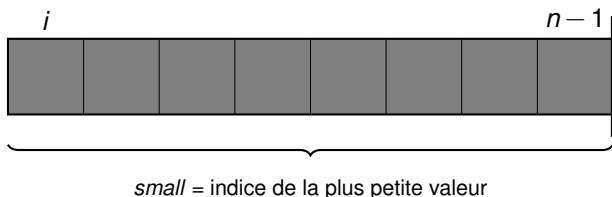


Remarques :

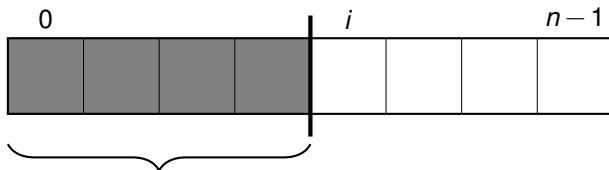
- Observez que le premier indice considéré est i (voyez-vous pourquoi ?)
- i est fixé (car i est l'indice de la boucle externe)
- la preuve de cet invariant est laissée en exercice : similaire à l'exemple 3 ci-avant, sauf qu'ici on cherche l'indice (et pas la valeur) du minimum (et pas du maximum)

Tri par sélection : invariant de la boucle interne

A la fin de la boucle interne, on peut donc considérer que `small` contient l'indice de la plus petite valeur de la zone de la liste qui commence à i et va jusqu'à la fin.



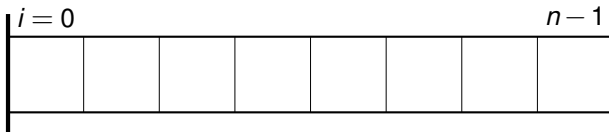
Tri par sélection : invariant de la boucle externe



la partie traitée est triée et toutes ses valeurs
sont plus petites que la partie non-traitée

Avant d'entrer dans la boucle externe

```
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])
```



La partie traitée est vide, et l'invariant est donc trivialement vrai.

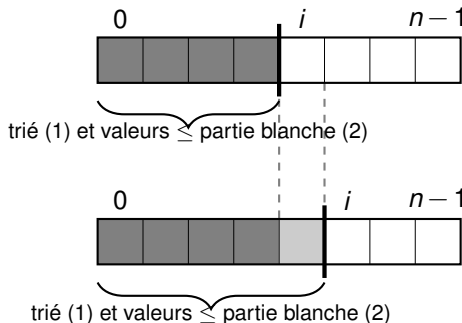
Après chaque nouvelle itération de la boucle externe

```

1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])

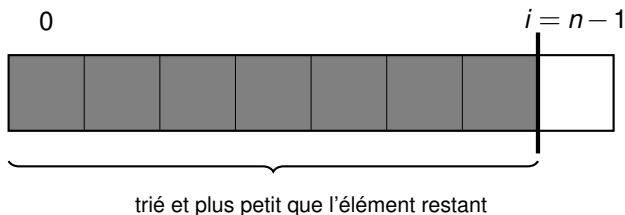
```

- on sait que la boucle interne nous donne l'indice (*small*) du plus petit élément de la partie blanche
- l'élément d'indice *small* est placé à l'indice *i*
- comme il est $\geq$ que les valeurs déjà traitées, cela reste trié et (1) est vrai
- comme il est le plus petit de la zone blanche, (2) restera vrai



Après l'exécution complète de la boucle externe

A la fin, $i = n - 1$ et on se retrouve, grâce à l'invariant de la boucle externe, dans la configuration suivante :



Les données sont donc complètement triées et ceci conclut la preuve d'exactitude du tri par sélection.